Practical- #4

Aim: WAP to add block of 8-bit numbers.

Algorithm :-

1. Start the Program and initlize the accumlator to 00H.
2. Set Counter of 5 numbers.
3. Load memory Pointer to starting address.
4. Add each numbers to accumlator.
5. If Carry occures, So Increment Carry Register.
6. Move to neat memory location and RePeat until Counter becomes zero.
7. Store the final answer.
8. Terminate the Program.

Program :-

```
        MVI A, 00H
        MVI C, 00H
        MVI D, 05H
        LXI H, 4150H
AGAIN:
        ADD M
        JNC NEXT
        INR C
```

Subject Name :

Subject Code :

B. Tech. ____Year ___ Semester ___

Annexure No.:

NEXT:

```
INX H
DCR D
JNZ AGAIN
STA 4156H
MOV A,C
STA 4157H
HLT.
```

Observation :-

Input :-

```
4150H = 01 H
4151H = 02 H
4152H = 03 H
4153H = 04 H
4154H = 05 H.
```

Output :-

```
4155H = 00 H
4156H = 0F H
```

$(15)_{10} \rightarrow (0F)_{16}$.

Enrollment No. :

Page No :

Annexure No.:

Memory

0x0 - 0x1A

0x2050 - 0x2053

0x4150 - 0x4156

Displaying Memory Locations from 0x4150 to 0x4156

Double click the value to edit then press Enter to save the value or Tab to edit the next location.

0x4150	01
0x4151	02
0x4152	03
0x4153	04
0x4154	05
0x4155	00
0x4156	0F

```
1   ;WAP To add block of 8-bit numbers [5 numbers starting 4150H]
2
3   MVI A, 00H
4   MVI C, 00H
5   MVI D, 05H
6   LXI H, 4150H
7   AGAIN:
8         ADD M
9         JNC NEXT
10        INR C
11  NEXT:
12        INX H
13        DCR D
14        JNZ AGAIN
15        STA 4156H
16        MOV A,C
17        STA 4157H
18  HLT
```

Conclusion :-

This Program use a loop and Counter to add memory block into accumulator ensuring accuracy by tracking overflows with the carry.

02/03/26.

Practical - 5

Part - A

Aim : Write a 8085 assembly lang Program to find minimum out of 8-bit numbers.

Algorithm :-

1. Load first number into register A.
2. Move it to register B.
3. Load second number into A.
4. Compare A with B.
5. if A < B -> A is minimum -> go to step 7
6. else move B into A.
7. Store Result.

Program :-

```
LDA 2051H
MOV B,A
LDA 2052H
CMP B
JC STORE
MOV A,B
STORE: STA 2053H
HLT.
```

Observation :-

Input :-

2051H : 10
2052H : 07

Output :-

2053H : 07

Memory

0x0 - 0xF

0x2051 - 0x2053

Displaying Memory Locations from 0x2051 to 0x2053

Double click the value to edit then press Enter to save the value or Tab to edit the next location.

0x2051	10
0x2052	07
0x2053	07

```
1   LDA 2051H
2   MOV B, A
3   LDA 2052H
4   CMP B
5   JC STORE
6   MOV A, B
7
8   STORE: STA 2053H
9   HLT
```

Conclusion :-

This Program use registers and logical instructions to find the minimum numbers outof 2 8-bit number.

Ramesh 20/04/26

Subject Name :

Subject Code :

B. Tech. ___ Year ___ Semester ___

Annexure No.:

Part -B

Aim : Assembly lang Program to find maximum outof 8-bit numbers.

Algorithm :-

1. Load first number into Register A.
2. Move it to Register B.
3. Load second number into Register A.
4. Compare A coith B.
5. if A > B -> A is maximum -> go to step 7
6. else move B into A.
7. Store the result.

Program :-

```
    LDA  2051H
    MOV  B,A
    LDA  2052H
CMP  B
 JNC  STORE
 MOV  A,B

 STORE :  STA  2053H

 HLT.
```

Subject Name :

Subject Code :

B. Tech. ___Year ___ Semester ___

Annexure No.:

Observation :-

InPut :-

2051H = 10

2052H : 07

OutPut :-

2053H : 10

Memory

0x0 - 0xF

0x2051 - 0x2053

Displaying Memory Locations from 0x2051 to 0x2053

Double click the value to edit then press Enter to save the value or Tab to edit the next location.

0x2051	10
0x2052	07
0x2053	10

```
1  LDA 2051H
2  MOV B, A
3  LDA 2052H
4  CMP B
5  JNC STORE
6  MOV A, B
7
8  STORE: STA 2053H
9  HLT
```

Conclusion :-

This Program uses registers & memory, JumPing instructions to find the maximum number out of 2 8-bit numbers.

20/04/26.

Enrollment No. :

Page No :

Scanned with OKEN Scanner

Subject Name :

Subject Code :

B. Tech. ____Year ___ Semester ___

Annexure No.:

Practical - 6

Part-A

Aim:- Write assembly lang Program to Sort data in assending order.

Algorithm :-

1. Start
2. Set Count of numbers.
3. Compare adjacent elements.
4. if first > second ⇝ Swap.
5. Repeat for all elements.
6. Repeat Passes until Sorted.
7. Stop.

Program :-

```
        MVI  C,05H
AGAIN:  MOV  D,C
        LXI  H,2050H
LOOP :  MOV  A,M
        INX  H
        CMP  M
        JC   NEXT
        MOV  B,M
        MOV  M,A
        DCX  H
        MOV  M,B
        INX  H
```

Enrollment No. :

Page No :

Scanned with OKEN Scanner

```
NEXT : DCR D
       JNZ LOOP
       DCR C
       JNZ AGAIN
       HLT.
```

Observation :-

Input :-

2050H : 05
2051H : 04
2052H : 03
2053H : 02
2054H : 01
2055H :

OutPut :-

2050H :
2051H : 01
2052H : 02
2053H : 03
2054H : 04
2055H : 05

Annexure No.:

Memory

0x0 - 0x19

0x2050 - 0x2055

Displaying Memory Locations from 0x2050 to 0x2055

ⓘ Double click the value to edit then press Enter to save the value or Tab to edit the next location.

0x2050	05
0x2051	04
0x2052	03
0x2053	02
0x2054	01
0x2055	00

```
1   MVI C, 05H
2
3   AGAIN: MOV D, C
4          LXI H, 2050H
5
6   LOOP:  MOV A, M
7          INX H
8          CMP M
9          JC NEXT
10         MOV B, M
11         MOV M, A
12         DCX H
13         MOV M, B
14         INX H
15
16  NEXT:  DCR D
17         JNZ LOOP
18         DCR C
19         JNZ AGAIN
20
21         HLT
```

0x2050	00
0x2051	01
0x2052	02
0x2053	03
0x2054	04
0x2055	05

Conclusion:

This Program uses adjecent comparision method and helps to sort data in assending order formate.

Enrollment No. :

20/04/26

Page No :

Part-B

Aim: Write a assembly lang Program to sort the data in desending order.

Algorithm:

1. Start.
2. Set Count of Numbers.
3. Compare adjacent element.
4. if first < Second -> Swap.
5. Repeat for all elements.
6. Repeat Passes until sorted
7. Stop

Program:

```
MVI  C, 05H

AGAIN: MOV D, C
       LXI H, 2050H.

LOOP: MOV A, M.
      INX H
      CMP M
      JNC NEXT.
      MOV B, M
      MOV M, A
      DCX H
      MOV M, B
      INX H.
```

```
NEXT: DCR D
      JNZ LOOP
      DCR C
      JNZ AGAIN.

HLT
```

Observation :

Input :

2050H :
2051H : 01
2052H : 02
2053H : 03
2054H : 04
2055H : 05

Output :

2050H : 05
2051H : 04
2052H : 03
2053H : 02
2054H : 01.
2055H :

Subject Code :

B. Tech. ____Year ___ Semester ___

Annexure No.:

Memory

0x0 - 0x19

0x2050 - 0x2055

Displaying Memory Locations from 0x2050 to 0x2055

ⓘ Double click the value to edit then press Enter to save the value or Tab to edit the next location.

0x2050	00
0x2051	01
0x2052	02
0x2053	03
0x2054	04
0x2055	05

```
1   MVI C, 05H
2
3   AGAIN: MOV D, C
4          LXI H, 2050H
5
6   LOOP:  MOV A, M
7          INX H
8          CMP M
9          JNC NEXT
10         MOV B, M
11         MOV M, A
12         DCX H
13         MOV M, B
14         INX H
15
16  NEXT:  DCR D
17         JNZ LOOP
18         DCR C
19         JNZ AGAIN
20
21  HLT
```

0x2050	05
0x2051	04
0x2052	03
0x2053	02
0x2054	01
0x2055	00

Conclusion:

This Program uses adjecent Comparision algo & helps to sort the data in desending order.

20/04/26.

Enrollment No. :

Page No :

Scanned with OKEN Scanner

Practical - 7

Part-A

Aim: Write an 8085 assembly lang Program to get the minimum from block of 8-bit numbers.

Algorithm:

1. Start
2. Load first number as minimum.
3. Compare with next number
4. if Smaller -> update minimum
5. Repeat for all numbers.
6. Store minimum.
7. Stop.

Program:

```
MVI  C,05H
LXI  H, 2051H
MOV  A,M.
INX  H.
DCR  C

AGAIN: CMP M
       JC  SKIP
       MOV A,M.

SKIP:  INX H
       DCR C
       JNZ AGAIN.
```

STA 2057 H.
HLT.

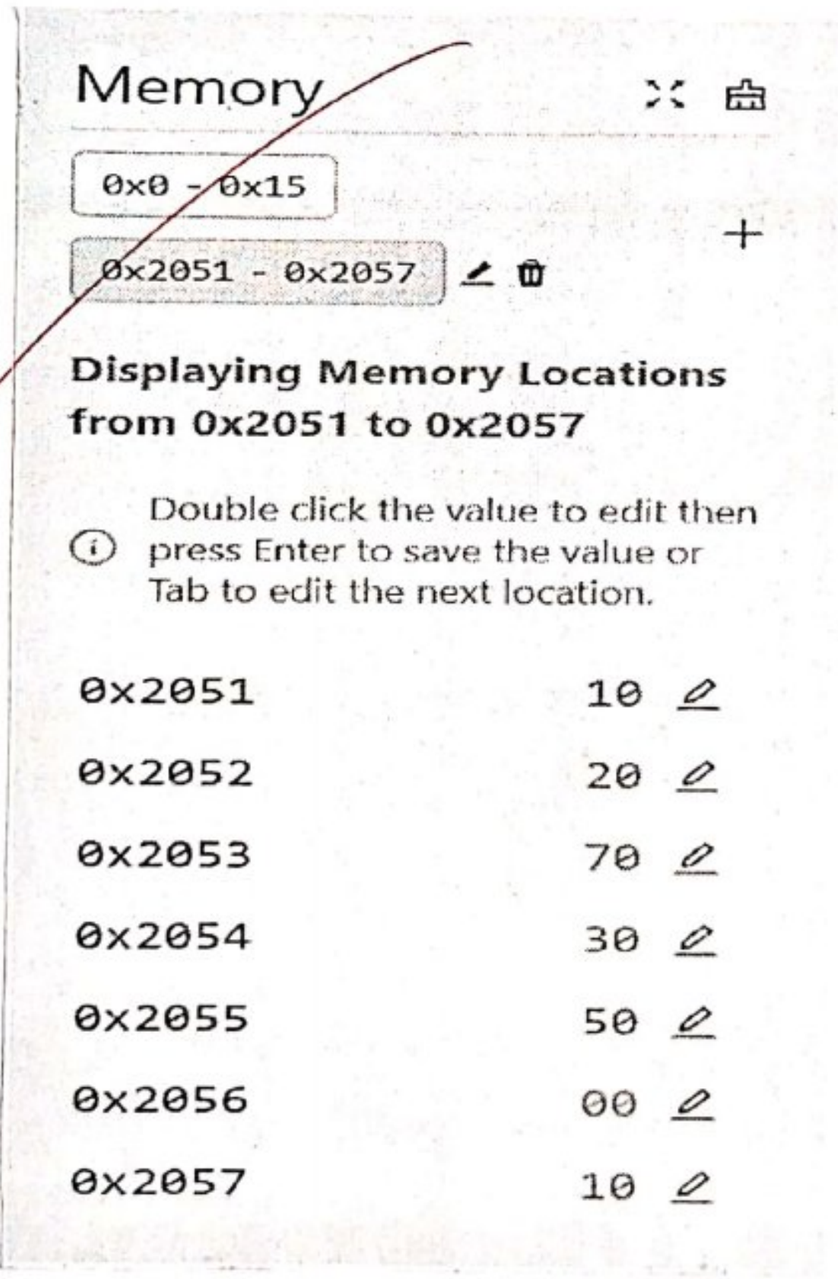
Observation :

Input :

2051H : 10
2052H : 20
2053H : 70
2054H : 30
2055H : 50

Output :

2056 H :
2057 H : 10




```
1   MVI C, 05H
2   LXI H, 2051H
3   MOV A, M
4   INX H
5   DCR C
6
7   AGAIN: CMP M
8          JC SKIP
9          MOV A, M
10
11  SKIP:  INX H
12         DCR C
13         JNZ AGAIN
14
15  STA 2057H
16  HLT
```

Conclusion :

This Program uses Jumping & memory instructions to find the minimum number out of n number of 8-bit block.

Enrollment No. :

20/04/26

Page No :

Part-B

Aim : Write an 8085 assembly lang Program to get the maximum from block of n 8-bit numbers.

Algorithm :

1. Load the first number as maximum
2. Compare with next number.
3. If greater ~> update maximum
4. Repeat for all numbers.
5. Store maximum.
6. Stop.

Program :

```
MVI  C,05H
LXI  H, 2051H.
MOV  A,M.
INX  H.
DCR  C

AGAIN: CMP M
       JNC SKIP
       MOV A,M.

SKIP:  INX H
       DCR C
       JNZ AGAIN.

       STA 2067H.

       HLT.
```

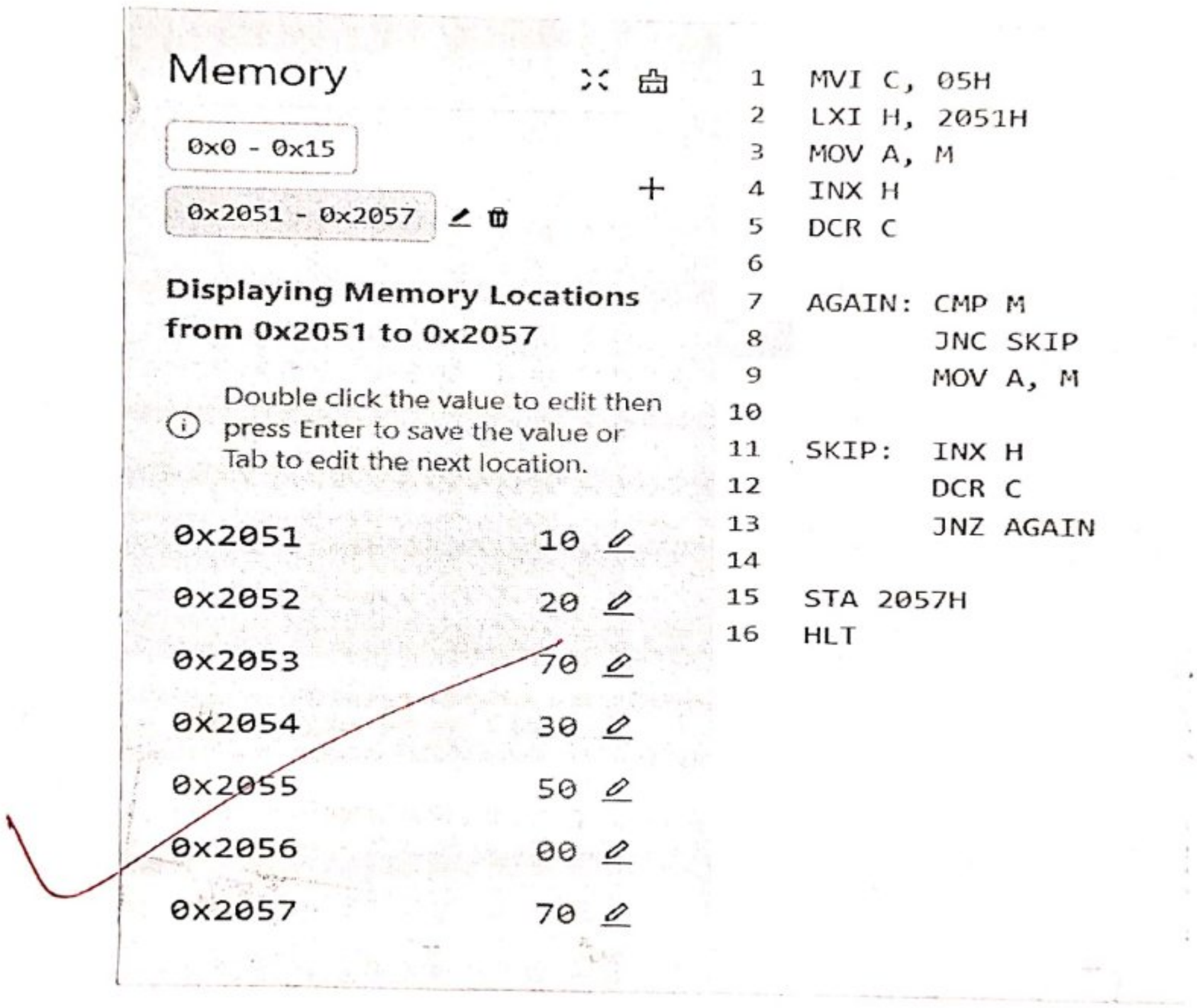
Observation :

Input :

2051H : 10
2052H : 20
2053H : 70
2054H : 30
2055H : 50

Output :

2056H :
2057H : 70



Conclusion :

This Program uses Jumping & memory instructions to find maximum number out of n number of Data of 8-bit block.

Parul
20/04/26

Enrollment No. :

Page No :

Practical-8

Aim: WAP to Convert BCD to Binary.

Algorithm:

1. Load BCD Number.
2. Separate unit Digit.
3. Separate tens digit.
4. Multiply tens by 10.
5. Add units.
6. Store result
7. Stop.

Program:

```
LDA  2005A
MOV  B,A

ANI  0FH
MOV  D,A

MOV  A,B
ANI  00F0H.
RLC
RLC
RLC
RLC.
```

```
MOV  E,A
MVI  C, 0AH
MVI  A, 00H

LOOP: ADD E
      DCR C
      JNZ LOOP

ADD  D

STA  2006H

HLT
```

Observation :-

Input :-

2005H : 27

Output :-

2006H : 1B

Subject Code :

B. Tech. ____Year ___ Semester ___

Annexure No.:

Memory

0x0 - 0x1C

0x2005 - 0x2006

Displaying Memory Locations from 0x2005 to 0x2006

Double click the value to edit then press Enter to save the value or Tab to edit the next location.

0x2005	27
0x2006	1B

```
1   LDA 2005H
2   MOV B, A
3
4   ANI 0FH
5   MOV D, A
6
7   MOV A, B
8   ANI 00F0H
9   RLC
10  RLC
11  RLC
12  RLC
13
14  MOV E, A
15  MVI C, 0AH
16  MVI A, 00H
17
18  LOOP: ADD E
19        DCR C
20        JNZ LOOP
21
22  ADD D
23  STA 2006H
24  HLT
```

Conclusion:

This Program helps to Convert our BCD Data into Binary formate

20/04/26

Enrollment No. :

Page No :

Scanned with OKEN Scanner

Practical - 9

Aim: Write a Program to Convert binary to BCD.

Algorithm:

1. Load binary number into A.
2. Initlize Counter.
3. Compare A with 10
4. if A < 10 -> go to step 7.
5. Subtract 10 from A.
6. Increment Counter.
7. Store the remainder in memory
8. Move Counter to A.
9. Store the result
10. Stop.

Program:

```
LDA 2005H.
MVI B,00H.

AGAIN: CPI 0AH.
       JC NEXT.
       SUI 0AH.
       INR B
       JMP AGAIN.
```

NEXT : STA 2006H.
MOV A,B
STA 2007H.

HLT

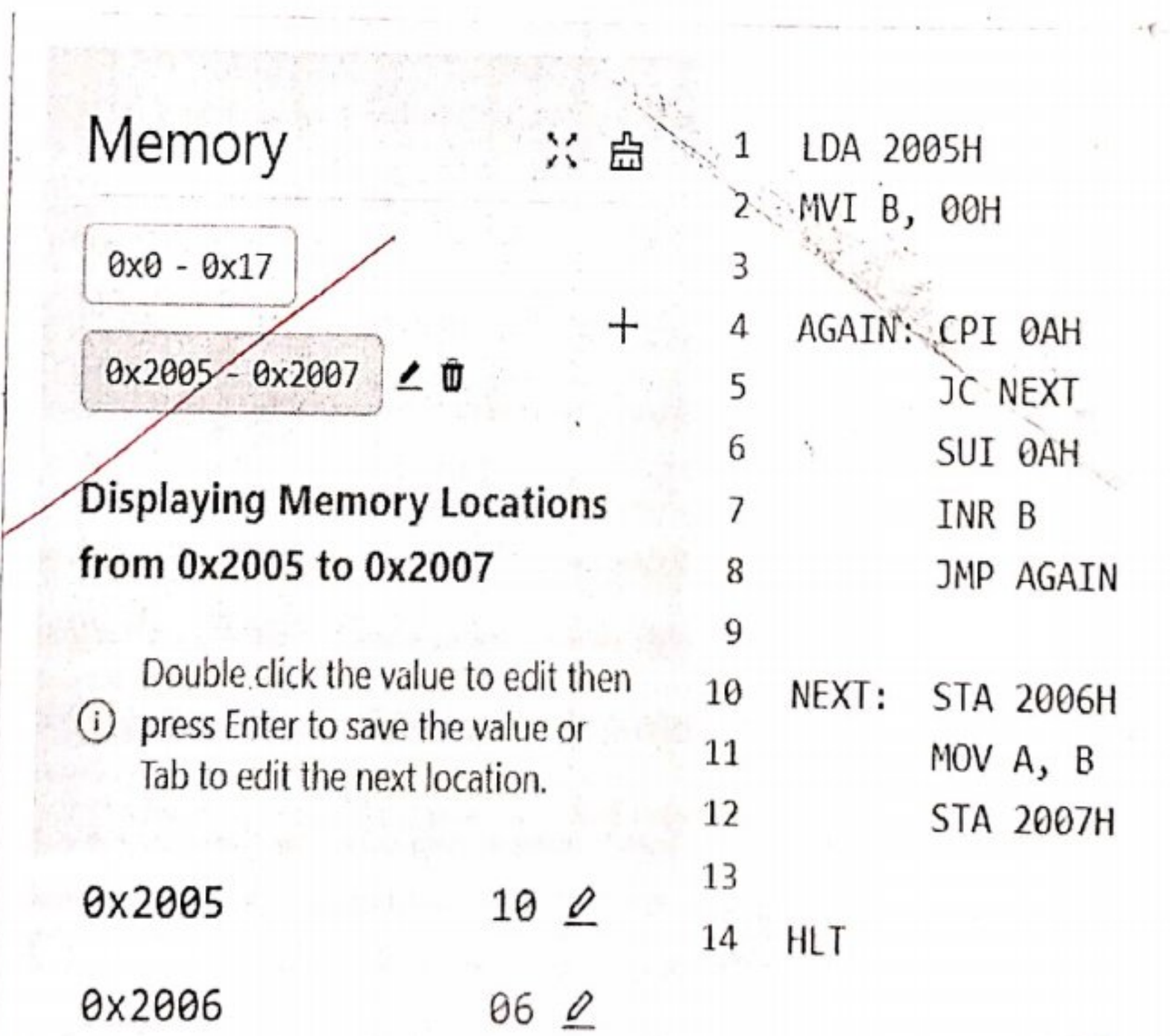
Observation :

Input :

2005H : 10

Output :

2006H : 06
2007H : 01.



Conclusion :
This Program helps to convert binary data into BCD formate.

20/04/26

Practical-1

Aim:

Part-A: Addition of two 8 bit numbers using 8085.

Algorithm:

1. Start the Program by loading the first data into Accumulator.
2. Move the data to a ~~ge~~ register (B).
3. Get the second data and load it into accumulator.
4. Add two register Contents.
5. Check for Carry.
6. Store the value of Sum and Carry in memory location.
7. Terminate the Program.

Program:

```
      MVI  C,00H.
      LDA  2150H.
      MOV  B,A.
      LDA  2151H.
      ADD  B
      JNC  LOOP
      INR  C
LOOP: STA  2152H
      MOV  A,C
      STA  2153H
      HLT
```

Observation :-

InPut :-

2150H : 88 H
2151H : 82 H

OutPut :-

2152H : 0A H
2153H : 01 H.

Part A

Memory

0x0 - 0x15

0x2050 - 0x2051

0x2150 - 0x2153

Displaying Memory Locations from 0x2150 to 0x2153

Double click the value to edit then press Enter to save the value or Tab to edit the next location.

0x2150	88
0x2151	82
0x2152	0A
0x2153	01

```
1   MVI C,00H
2    LDA 2150H
3    MOV B,A
4    LDA 2151H
5    ADD B
6    JNC LOOP
7    INR C
8   LOOP: STA 2152H
9    MOV A,C
10   STA 2153H
11   HLT
```

Conclusion :-

We learnt to Perform addition of two 8-bit numbers in 8085 simulator.

02/03/26

Part-B :- WAP to Add two 16-bit numbers stored in register.

Algorithm :-

1. Start Program by loading first data into accumulator.
2. Move data to Register.
3. Get the second data and load it into the accumulator.
4. Substract the 2nd register content.
5. Check for borrow.
6. Store the difference & borrow in memory location
7. Terminate the Program.

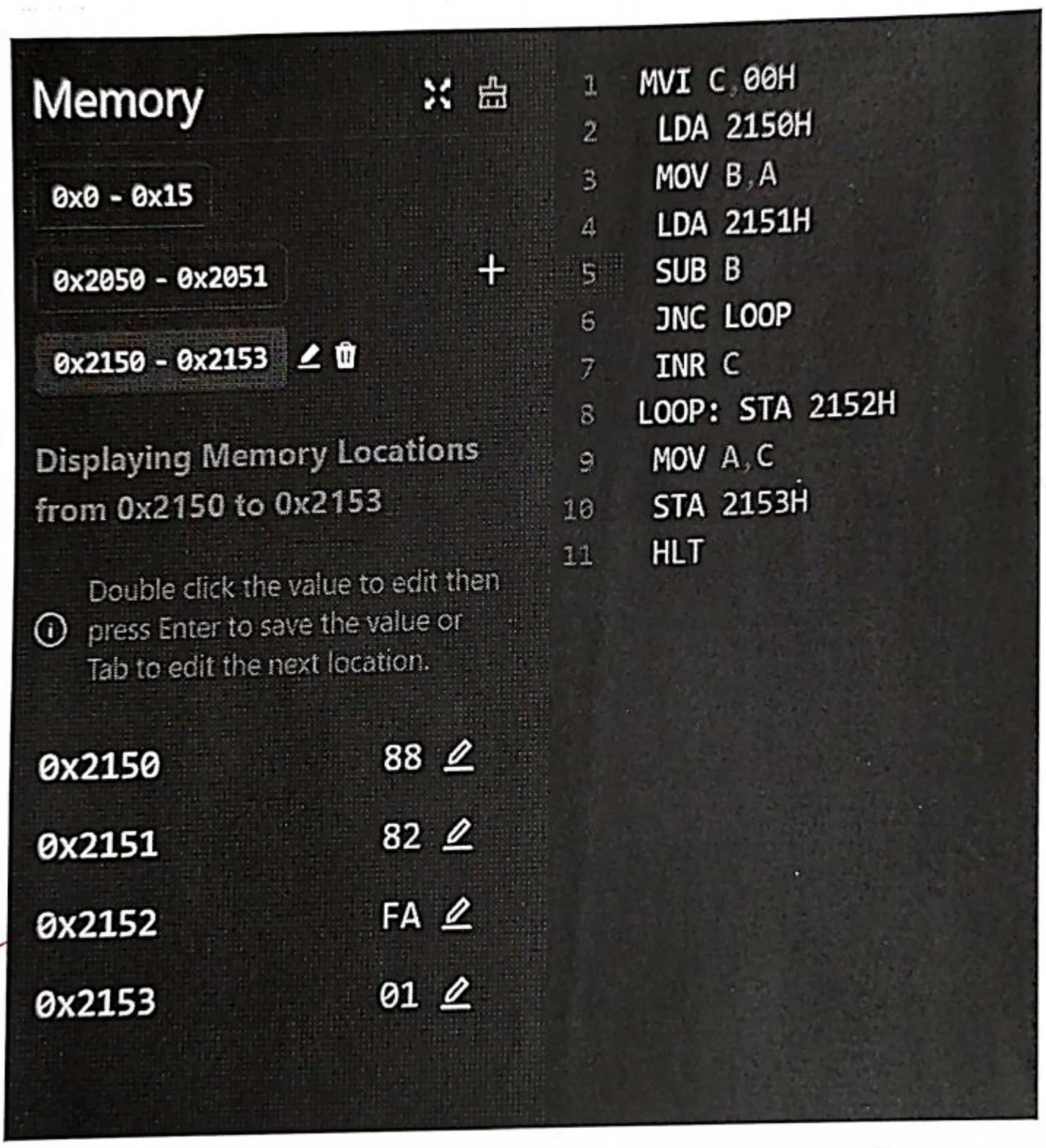
Program :

```
      MVI  C,00H.
      LDA  2150H.
      MOV  B,A
      LDA  2151H.
      SUB  B
      JNC  LOOP
      INR  C
LOOP: STA  2152H
      MOV  A,C
      STA  2153H
      HLT
```

Observation :=

Input :- 2150 H : 88 H
2151 H : 82 H

Output :- 2152 H : FA H
2153 H : 01 H.



Conclusion :=

In this Practical we learnt to Perform Substraction of two 8-bit numbers in 8085 Simulator.

02/03/26.

Practical-02

Aim: WAP to Perform addition of 16-bit numbers using 8085.

Software Required :- Simulator 8085.

Algorithm :-

1. Lower Part of number is stored in accumulator.
2. Move value of accumulator into Register.
3. The lower Part of 2nd number is stored in accumulator.
4. Add lower bits of both numbers.
5. The reasult will be stored in a memory location.
6. The UPPer Part of number will be stored in accumulator.
7. Move Content of Register A to Register B.
8. The UPPer Part of number is stored in accumulator.
9. Check Carry.
10. Store UPPer byte & Carry into memory locations.

Program :=

```
        MVI  C,00H
        LDA  2051H
        MOV  B,A
        LDA  2054H.
        ADD  B.
        STA  2053H.
        LDA  2052H
        MOV  D,A.
        LDA  2055H
        ADC  D
        JNC  LOOP
        INR  C
LOOP:   STA  2056H
        MOV  A,C
        STA  2057H

        HLT.
```

Observation :=

InPut :- 2051H : 80H
2055H : 20H
2053H : 81H
2054H : 81H

OutPut :- 2053H : A0H
2056H : 02H
2057H : 01H.

Memory

0x2051 - 0x2057

Displaying Memory Locations from 0x2051 to 0x2057

Double click the value to edit then press Enter to save the value or Tab to edit the next location.

0x2051	80
0x2052	81
0x2053	A0
0x2054	20
0x2055	81
0x2056	02
0x2057	01

```
1   MVI C,00H
2   LDA 2051H
3   MOV B,A
4   LDA 2054H
5   ADD B
6   STA 2053H
7   LDA 2052H
8   MOV D,A
9   LDA 2055H
10  ADC D
11  JNC LOOP
12  INR C
13  LOOP: STA 2056H
14        MOV A,C
15        STA 2057H
16  HLT
```

Conclusion :-

In this Practical we learnt to Perform addition of two 16-bit numbers in 8085 Simulator.

Pawar
02/03/26.

Practical-3

Aim:

Part-A WAP to Perform multiplication of 2- 8 bit numbers using 8085.

Algorithm :=

1. Start the Program & initlize Registers.
2. Load the first number into Register B.
3. Load second number into Register C.
4. Add register B to accumulator.
5. If Carry occures, so implement Register D.
6. Repeat addition until Counter becomes zero.
7. Store result in memory locations and terminate the Program.

Program :=

```
MVI D,00H
MVI A,00H
LXI H, 4150H
MOV B, M
INX H
MOV C, H

LOOP: ADD B
      JNC NEXT
      INR D.
```

```
NEXT: DCR C
      JNZ LOOP.

      STA 4152H.
      MOV A,D.
      STA 4153H
      HLT.
```

Observation :-

InPut :=

4150H : 1B H
4151H : 0C H

outPut :=

4152H : 44 H
4153 H : 01 H.

Conclusion :-

In this experiment we learnt to Perform multiplication of two 8-bit numbers using 8085.

Tanvi B
02/03/26.

PART A: MULTIPLICATION

Memory

0x0 - 0x1A

0x2050 - 0x2053

0x4150 - 0x4153

Displaying Memory Locations from 0x4150 to 0x4153

ⓘ Double click the value to edit then press Enter to save the value or Tab to edit the next location.

0x4150	1B
0x4151	0C
0x4152	44
0x4153	01

```
1   MVI D, 00H
2   MVI A, 00H
3   LXI H, 4150H
4   MOV B, M
5   INX H
6   MOV C, M
7
8   LOOP: ADD B
9         JNC NEXT
10        INR D
11
12  NEXT: DCR C
13        JNZ LOOP
14
15  STA 4152H
16  MOV A, D
17  STA 4153H
18  HLT
```

Part B:

WAP to Perform division between 2 numbers using 8085.

Algorithm :=

1. Start the Program and load dividend from memory location 2050H.
2. Move the next memory location and load divisor into ~~Resi~~ second Register.
3. Clear Register C to store quotient.
4. Compare dividend with divisor.
5. If divided is less than divisor go to end.
6. else Subtract divisor form dividend & increment quotient.
7. Store reminder and quotient in memory locations
8. terminate the Program:

Program :=

```
LXI H, 2050H.
MOV A, M.
INX H
MOV B, M.
MVI C, 00H
```

Subject Code :

B. Tech. ____Year ___ Semester ___

Annexure No.:

```
DIV_LOOP:
        CMP B
        JC DIV_END
        SUB B
        INR C
        JMP DIV_LOOP.

DIV_END:
        STA 2053H
        MOV A, C
        STA 2054H
        HLT.
```

Observation :=

Input :-

2050H : 09H

2051H : 5AH

Output :-

2052H : 00H

2053H : 09H.

Enrollment No. :

Page No :

Scanned with OKEN Scanner

PART B: DIVISION

Memory

0x0 - 0x18

0x2050 - 0x2053

Displaying Memory Locations from 0x2050 to 0x2053

Double click the value to edit then press Enter to save the value or Tab to edit the next location.

0x2050	09
0x2051	5A
0x2052	00
0x2053	09

```
1  ; Program to divide two 8-bit numbers
2  ; Dividend at 2050H, Divisor at 2051H
3  ; Quotient stored at 2053H, Remainder at 2054H
4
5        LXI H, 2050H  ; Point to dividend
6        MOV A, M      ; Load dividend into A
7        INX H         ; Point to divisor
8        MOV B, M      ; Load divisor into B
9        MVI C, 00H    ; Clear quotient register (C)
10
11 DIV_LOOP:
12       CMP B         ; Compare A and B (A - B)
13       JC DIV_END    ; If A < B (Carry=1), division is done
14       SUB B         ; A = A - B
15       INR C         ; Increment quotient (C)
16       JMP DIV_LOOP  ; Repeat
17
18 DIV_END:
19       STA 2053H     ; Store remainder (A) in 2053H
20       MOV A, C      ; Move quotient to A
21       STA 2054H     ; Store quotient in 2054H
22       HLT           ; Terminate
23
```

Conclusion :-

In this experiment coe learnt to Perform division of two 8-bit numbers using 8085.

Tanvi
02/03/26.